UNIVERSIDAD DE ANTIOQUIA

Facultad de Ingeniería

Informática II — Semestre 2026-1

Desafío II

UdeAWorldCup

Simulación del Mundial de Fútbol FIFA 2026

Estudiante

Oscar Trujillo

Profesor

Aníbal Guerra

24 de abril de 2026

1. Introducción

UdeAWorldCup es una simulación del Mundial de Fútbol FIFA 2026, desarrollado en C++ bajo la programación orientada a objetos, el sistema gestiona los 48 equipos clasificados, sus plantillas de 26 jugadores y los 104 partidos del torneo (72 de fase de grupos y 32 eliminatorios) aplicando probabilidad para generar resultados realistas y actualizar las estadísticas históricas en tiempo de ejecución.

La aplicación se interactúa por consola a través de un menú con cinco funcionalidades principales: carga de datos desde un archivo CSV, conformación de bombos y sorteo de grupos, simulación completa del torneo, generación de un informe estadístico y medición del consumo de recursos. Al final el sistema persiste los datos actualizados de los 1.248 jugadores en un archivo de salida.

Este informe documenta el análisis del problema, las decisiones de diseño aplicadas, los algoritmos esenciales del sistema y los retos enfrentados durante el desarrollo.

2. Análisis del problema

Al estudiar el enunciado identifiqué cinco funcionalidades centrales que se debían cubrir, todas accesibles desde un menú independiente:

1. Carga y actualización de datos desde un archivo CSV proporcionado con persistencia posterior de las estadísticas individuales de los jugadores.
2. Conformación de los 12 grupos del torneo respetando las restricciones del sorteo, el país anfitrión va directo al bombo 1, los demás equipos se distribuyen por ranking FIFA y ningún grupo puede tener más de un equipo de la misma confederación, salvo UEFA con un máximo de dos.
3. Simulación de los partidos en cada etapa, aplicando la fórmula de Poisson de la guía para los goles esperados y las probabilidades específicas para faltas, tarjetas amarillas, rojas y goles individuales.
4. Generación de un informe estadístico final con el ranking de los cuatro primeros puestos, los máximos goleadores, el equipo con más goles históricos tras el torneo, y la confederación con mayor presencia por etapa eliminatoria.
5. Medición del consumo de recursos en cada funcionalidad, reportando iteraciones realizadas y memoria estimada en uso al cerrar cada operación.

El enunciado establece varias simplificaciones que decidí adoptar tal cual para no sobrecargar el modelo, una sede única identificada como nombreSede, tres árbitros con códigos fijos “codArbitro1” a “codArbitro3”, hora fija 00:00 para todos los partidos, y una fabricación artificial de los jugadores con etiquetas tipo “nombre1”, “apellido1” hasta “nombre26”.

Las restricciones técnicas más relevantes del desarrollo son, la prohibición de usar contenedores de la STL, structs, y librerías de terceros, uso obligatorio de memoria dinámica en el núcleo del sistema, diseño multiarchivo separado en .h y .cpp y aplicación de los conceptos OOP estudiados en el curso, incluyendo plantillas, sobrecarga de operadores, funciones amigas y la Regla de los Tres.

3. Diseño de la solución

3.1. Diagrama de clases UML

El diagrama UML completo se entrega como archivo adjunto al informe. Contiene 13 clases del dominio más una plantilla genérica Lista<T>. La clase orquestadora es Torneo, que coordina el flujo completo del sistema.

Las relaciones más importantes entre clases son:

- Composición fuerte: Equipo contiene una Lista<Jugador> por valor (los jugadores viven y mueren con el equipo) de forma similar, Partido contiene dos EstadisticaPartido por valor, una por cada equipo.
- Agregación: Equipo referencia su Confederación mediante un puntero ya que varias selecciones comparten una misma confederación que vive aparte en el Torneo. Lo mismo aplica para los punteros a Equipo dentro de Partido y Grupo.
- Asociación temporal: Bombo es un contenedor temporal de punteros a Equipo que se vacía durante el sorteo, una vez completado el sorteo, los bombos quedan vacíos pero los equipos siguen vivos en el Torneo.

3.2. Conceptos OOP aplicados

El sistema aplica los conceptos OOP exigidos por la guía de la siguiente manera:

Concepto	Dónde se aplica
Encapsulación	Todos los atributos de las 13 clases son privados; el acceso se hace por getters y setters.
Composición	Equipo posee Lista<Jugador> por valor, Partido posee dos EstadisticaPartido.
Agregación	Equipo apunta a Confederación, Partido apunta a sus dos Equipo.
Sobrecarga de operadores	operator< y operator== en Equipo; operator+= en EstadisticaEquipo y EstadisticaJugador, operator[] y operator= en Lista, operator<< como función amiga en clases imprimibles.
Funciones amigas	operator<< se declara friend porque el operando izquierdo (std::ostream) no es miembro de la clase.
Plantillas	Lista<T> es una plantilla propia instanciada con int, Jugador, Equipo*, Partido*, Grupo*, Confederacion*, RegistroJugadorPartido y otros tipos.
Regla de los Tres	Implementada completa en Lista<T> (constructor de copia, operator=, destructor con copia profunda).
Prevención de copia accidental	Torneo declara constructor de copia y operator= como =delete para evitar dobles liberaciones de memoria por copia inadvertida.

4. Estructuras de datos

4.1. Plantilla Lista<T> propia

Como el enunciado prohíbe el uso de contenedores STL diseñé una plantilla genérica Lista<T> basada en arreglo dinámico redimensionable. La plantilla maneja internamente un puntero a un arreglo de tipo T, junto con dos contadores que sontamaño actual y capacidad reservada. Cuando la lista se llena, redimensiona duplicando la capacidad para mantener un costo amortizado constante en las inserciones.

La elección de un arreglo dinámico sobre una lista enlazada se justifica porque en la mayoría de los usos del sistema los tamaños son conocidos y fijos, 48 equipos, 12 grupos, 4 equipos por grupo, 26 jugadores, 11 convocados. En estos casos, el acceso por índice O(1) y la mejor localidad de caché del arreglo dinámico superan ampliamente a la lista enlazada que ofrecería ventajas solo en escenarios con inserciones intermedias frecuentes que aquí no ocurren.

4.2. Instanciaciones de Lista<T> en el sistema

Lugar	Instancia	Justificación
Torneo	Lista<Equipo*>	Tamaño fijo (48). Se ordena por ranking FIFA al conformar bombos.
Torneo	Lista<Confederacion*>	Tamaño pequeño (6). Se consulta por código durante la carga.
Torneo	Lista<Grupo*>	Tamaño fijo (12). Recorrido secuencial.
Torneo	Lista<Partido*>	Crece a 104 (72 + 32). Recorrido secuencial.
Equipo	Lista<Jugador>	Tamaño fijo (26). Acceso aleatorio para escoger 11 convocados.
Grupo	Lista<Equipo*>	Tamaño fijo (4).
Bombo	Lista<Equipo*>	Tamaño fijo (12). Se vacía durante el sorteo.
EstadisticaPartido	Lista<RegistroJugadorPartido>	Tamaño fijo (11) por valor.

4.3. Paso por valor vs. referencia

Los tipos primitivos (int, bool, double, char) se pasan por valor porque la copia es trivial. Los objetos estructurados grandes (Equipo, Partido, Lista<T>) se pasan por referencia constante para evitar copias costosas. Cuando varias colecciones comparten una misma entidad (por ejemplo, varios equipos asignados a la misma confederación) se usan punteros en lugar de copias.

5. Algoritmos esenciales

Esta sección describe la la lógica de los procedimientos no triviales del sistema. La intradocumentación detallada acompaña cada función en el código fuente.

5.1. Carga del archivo CSV

El parser abre el archivo en modo binario para detectar correctamente el BOM UTF-8 al inicio (tres bytes EF BB BF). Si están presentes, los descarta, después salta dos líneas iniciales (título y cabecera) y procesa cada línea restante separándola por punto y coma, antes de comparar la confederación pasa el texto a mayúsculas para unificar casos como “Concacaf” con “CONCACAF”. Para cada equipo nuevo busca la confederación en una lista interna, si ya existe la reutiliza sino la crea nueva. El código de país se asigna mediante una tabla manual con los códigos FIFA estándar de tres letras (FRA, ESP, ARG, etc.), con un fallback automático que toma las tres primeras letras en mayúsculas si el país no aparece en la tabla.

5.2. Fabricación artificial de jugadores

Tras cargar cada equipo, el sistema crea sus 26 jugadores con números de camiseta del 1 al 26, nombres “nombre1” hasta “nombre26” y apellidos “apellido1” hasta “apellido26”. Los goles históricos del equipo se reparten uniformemente, si el equipo tiene G goles totales cada jugador recibe G÷26 goles y los primeros (G mod 26) jugadores reciben un gol adicional para que la suma cuadre exactamente, el resto de las estadísticas (partidos jugados, minutos, tarjetas, faltas, asistencias) inicia en cero.

5.3. Sorteo de bombos y grupos

El sorteo se realiza en dos pasos, primero, se conforman los cuatro bombos, el equipo anfitrión que es estados unidos va directo al bombo 1, los 47 equipos restantes se ordenan por ranking FIFA ascendente mediante un insertion sort manual y se distribuyen en grupos de 11 junto a USA, 12, 12 y 12 al bombo 1, 2, 3 y 4 respectivamente, elegí insertion sort porque es estable, in-place y trivial de auditar a esta escala (n=47) soluciones más sofisticadas como quicksort no aportarían beneficio práctico.

Segundo se sortean los 12 grupos, para cada uno se extrae aleatoriamente un equipo de cada bombo validando dos restricciones, ningún grupo puede repetir confederación (excepto UEFA con un máximo de dos equipos). Si la validación falla el equipo se devuelve al bombo y se intenta otro, como garantía si después de 200 intentos un slot no logra llenarse se ejecuta un swap con un equipo previamente asignado a un grupo anterior que sí sea compatible con el grupo actual, en las pruebas realizadas, el swap nunca tuvo que activarse.

5.4. Asignación de fechas en fase de grupos

Los 72 partidos de fase de grupos deben distribuirse en los 19 días disponibles entre el 20 de junio y el 8 de julio de 2026 con dos restricciones simultáneas, máximo 4 partidos por día y mínimo 3 días de descanso entre dos partidos del mismo equipo, el algoritmo es greedy, por cada partido recorre los 19 días en orden y asigna el primer día válido, si no encuentra ninguno aplica una relajación documentada que ignora la restricción de descanso siempre que el día tenga menos de 4 partidos. En las corridas observadas el greedy logra asignar los 72 partidos sin necesidad de relajaciones.

5.5. Simulación de un partido

La simulación de un partido individual sigue exactamente la fórmula del enunciado para calcular los goles esperados de cada equipo. Los promedios GF y GC se toman de las estadísticas históricas actualizadas de cada equipo.

Una vez calculados los λ de ambos equipos el algoritmo selecciona aleatoriamente 11 jugadores de los 26 mediante un Fisher-Yates manual sobre un arreglo de índices, para cada jugador convocado se simulan eventos en orden, faltas (con probabilidades 13%, 2.75% y 0.7% para la primera, segunda y tercera) tarjetas amarillas (6% para la primera, 1.15% para la segunda, dos amarillas suman una roja) y goles (el ciclo recorre los 11 jugadores con probabilidad 4% cada uno hasta que el equipo alcance su λ redondeado con un tope de 200 iteraciones para prevenir bucles infinitos)

La posesión del balón se calcula proporcionalmente al ranking inverso, el equipo de mejor ranking (número menor) recibe mayor posesión. Tras la simulación se actualizan las estadísticas históricas de los dos equipos y de los 22 jugadores convocados.

5.6. Prórroga en eliminatorias

Si un partido eliminatorio termina empatado se activa el bloque de prórroga, se identifica el equipo con mejor ranking FIFA y se incrementa su λ en uno, después se vuelven a simular los goles, si el desempate falla se reintenta hasta cinco veces, en el caso límite que ninguna funcione se otorga la victoria al equipo con mejor ranking directamente. Los minutos de los 22 jugadores convocados se actualizan a 120 cuando el partido entra en prórroga.

5.7. Clasificación y emparejamiento R16

Al finalizar la fase de grupos se construye la tabla de cada grupo aplicando los criterios de desempate de la guía, puntos descendente, diferencia de goles descendente, goles a favor descendente y como última instancia un sorteo aleatorio. De los 12 grupos clasifican los 12 primeros, los 12 segundos, y los 8 mejores terceros (ordenados por los mismos criterios).

El emparejamiento de R16 sigue las tres reglas de la guía, ocho de los doce primeros se enfrentan a los ocho terceros clasificados, los cuatro primeros restantes se enfrentan a los cuatro peores segundos y los ocho mejores segundos se emparejan entre sí. La restricción crítica es que ningún partido puede ser entre equipos del mismo grupo de la fase anterior, el algoritmo usa intentos aleatorios con validación, si tras 200 intentos no encuentra una pareja válida, ejecuta un swap entre partidos ya emparejados.

5.8. Etapas eliminatorias posteriores

Las etapas R16, R8 (octavos), QF (cuartos), SF (semifinales), tercer puesto y final se simulan en cascada. Tras cada etap los ganadores se emparejan secuencialmente para la siguiente. En semifinales, los dos perdedores van al partido del tercer puesto, mientras los dos ganadores van a la final. Todos los partidos de las etapas posteriores tienen fecha fija 10 de julio de 2026 según el enunciado y todos se simulan con la lógica de prórroga activada.

5.9. Medidor de recursos

La clase MedidorRecursos lleva dos contadores: iteraciones realizadas y memoria estimada en uso. Cada clase con memoria dinámica notifica al medidor cuando reserva o libera bytes y los bucles críticos llaman a registrarIteracion en cada ciclo. Al cerrar cada funcionalidad el sistema imprime un reporte con ambos

valores y reinicia el contador de iteraciones mientras que la memoria persiste porque refleja el estado actual del sistema.

6. Formato del archivo de jugadores

El enunciado pide diseñar un formato propio para persistir las estadísticas históricas de cada jugador. Opté por un archivo de texto plano con separador punto y coma y una línea por jugador con los campos en el siguiente orden:

codigoEquipo;numeroCamiseta;nombre;apellido;partidosJugados;goles;minutos;asistencias;amarillas;rojas;faltas

Un ejemplo de línea real generada por el sistema:

FRA;1;nombre1;apellido1;3;2;270;0;0;0;0

Justificación de estas decisiones:

- Texto plano: me permite inspección y depuración manual con cualquier editor, sin requerir librerías o herramientas externas.
- Punto y coma como separador: evita colisiones con posibles comas internas en nombres y mantiene el formato CSV compatible.
- Prefijo codigoEquipo: deja reconstruir la relación con el equipo al leer, sin necesidad de un archivo separado de mapeo.
- No se usa JSON ni XML porque requerirían librerías externas o un parser propio que añadiría complejidad sin valor real.

El archivo generado contiene 48 equipos × 26 jugadores.

7. Problemas en el desarrollo

Presento aquí los retos técnicos más relevantes y cómo los resolví:

- BOM UTF-8 en el archivo CSV: el archivo proporcionado tiene tres bytes invisibles al inicio (EF BB BF). Sin manejarlos el primer campo de la primera fila quedaba con caracteres invisibles que rompían la conversión a entero. Tuve que abrir el archivo en modo binario y saltar los tres bytes manualmente antes de empezar a parsear.
- Códigos de país no incluidos en el CSV: el archivo solo trae los nombres completos de los países . Mantuve una tabla manual con los códigos FIFA estándar de tres letras y un mecanismo de fallback que toma las tres primeras letras en mayúscula si el país no aparece en la tabla.
- Confederaciones inconsistentes: el CSV mezcla “UEFA” en mayúsculas con “Concacaf” en formato mixto. Sin normalización, la regla de máximo dos UEFA por grupo habría fallado. Resolví aplicando uppercase a todas las cadenas antes de comparar.
- Riesgo de doble liberación de memoria en Torneo: la clase Torneo posee múltiples colecciones de punteros con new. Una copia accidental de un objeto Torneo habría causado un doble delete en el destructor, para prevenirlo declaré el constructor de copia y el operator= como =delete, deshabilitándolos a nivel de compilación.
- Restricción de tres días entre partidos del mismo equipo: con un greedy puro existía el riesgo de no encontrar fecha válida para los últimos partidos, documenté una relajación opcional que ignora la restricción de descanso si el día tiene menos de 4 partidos pero en la práctica el greedy logró asignar los 72 partidos sin necesidad de aplicarla.
- Validación incorrecta inicial del filtro de terceros: en una primera versión del código de verificación asumí que las posiciones 9 a 12 del ranking de terceros eran las que no clasificaban. Al detectar que mi propio sort interno aplica un sorteo aleatorio como cuarto criterio de desempate, corregí la verificación para comparar estadísticas: ningún tercero eliminado debe tener stats estrictamente mejores que un clasificado.

8. Conclusiones

El sistema UdeAWorldCup cumple las cinco funcionalidades requeridas por el enunciado y aplica de forma efectiva los conceptos de de programación orientada a objetos, encapsulación, composición, agregación, plantillas, sobrecarga de operadores, funciones amigas y la regla de los tres.

La plantilla propia Lista<T> permitió cumplir la restricción de no usar contenedores STL sin sacrificar legibilidad ni eficiencia. La clase orquestadora Torneo libera correctamente toda la memoria dinámica al cerrarse y las copias accidentales se previenen explícitamente con =delete.

Una observación interesante de la simulación es el efecto de los promedios históricos sobre los resultados: equipos pequeños con muy pocos partidos clasificatorios pero buenos números (por ejemplo, Tunisia con un excelente promedio defensivo) pueden ganar el torneo en algunas corridas. Esto no es un bug sino una propiedad emergente de aplicar la fórmula del enunciado sobre datos reales de FIFA, donde la base estadística no está balanceada entre selecciones grandes y pequeñas.

Lecciones aprendidas: la importancia del diseño previo (UML) antes de implementar, el manejo cuidadoso de memoria dinámica en C++, el valor de las restricciones del enunciado para mantener el alcance acotado y la utilidad de pruebas pequeñas e incrementales después de cada bloque grande de código.

9. Referencias

FIFA. (2024). Revised 2023–2026 budget.

Inside World Football. (2025). FIFA reports $2bn upward budget revision to $13bn for 2023–26 cycle.

Sports Value. (2026). The 2026 FIFA World Cup will be the most lucrative in history.

Documentación oficial de C++17, cppreference.com.

Material del curso Informática II, Universidad de Antioquia, Semestre 2026-1.

10. Enlaces del proyecto

Repositorio de código fuente: https://github.com/Ostras21/UDEA-WorldCup

Video Youtube